

Practical No. 1

Definition : Write a program to demonstrate basic input and output operations in Python.

Aim/ Objective:

- To understand how to accept input from the user in Python.
- To display output using the print() function.
- To perform basic operations on user-provided data.
- To understand basic Python syntax for input and output.

Source Code:

```
name=input('Enter name:')  
num=int(input('Enter number:'))  
print(name)  
print(num)
```

Output:

```
Enter name:priyanshu  
Enter number:31  
priyanshu  
31
```

Conclusion:

The program successfully demonstrates the basic input and output operations in Python. It shows how to accept data from the user using input(), convert input into appropriate data types, store values in variables, perform a basic arithmetic operation, and display the results using print()

Practical No. 2

Definition : Write a program to illustrate the use of different data types and type casting.

Aim/ Objective:

- To understand different data types available in Python.
- To learn how to declare and use different types of data.
- To understand the concept of type casting in Python.
- To convert data from one type to another using type casting functions.
- To perform basic operations using different data types.

Source Code:

```
val_i=1234
```

```
val_f=90.00
```

```
val_s="abc"
```

```
print(type(val_i))
```

```
print(type(val_f))
```

```
print(type(val_s))
```

```
print("Type casting")
```

```
cast_i=int(val_f)
```

```
print("float to int",cast_i,type(cast_i))
```

```
cast_f=float(val_i)
```

```
print("int to float",cast_f,type(cast_f))
```

Output:

```
<class 'int'>
```

```
<class 'float'>
```

```
<class 'str'>
```

```
Type casting
```

```
float to int 90 <class 'int'>
```

```
int to float 1234.0 <class 'float'>
```

Conclusion:

The program successfully demonstrates the use of different data types and type casting in Python. It shows how to use various data types such as integer, float, string, and boolean, and how to convert data from one type to another using type casting functions. It helps in understanding how Python handles different types of data and performs basic operations on them.

Practical No. 3

Definition :Write a program to perform arithmetic relational and logical operations using Python operators.

Aim/ Objective:

- To understand the use of arithmetic, relational, and logical operators in Python.
- To perform basic arithmetic calculations using Python operators.
- To compare values using relational operators.
- To understand and perform logical operations using logical operators.
- To understand how different Python operators are used in programs.

Source Code:

```
"operator"

a=int(input('Enter number 1:'))
b=int(input('Enter number 2:'))
print('arithmatic operation:')
print('addition',a+b)
print('division',a/b)
print('multiplication',a*b)
print('substaction',a-b)

print('relational operatoin:')
if(a==b):
    print('a and b is same')
else:
    print('a and b are not same')
```

```
print('logical operation:')
if(a>b and b<a):
    print('this condition is right')
else:
    print('this is wrong')
```

Output:

Enter number 1:10

Enter number 2:20

arithmetic operation:

addition 30

division 0.5

multiplication 200

substaction -10

relational operatoin:

a and b are not same

logical operation:

this is wrong

Conclusion:

The program successfully demonstrates arithmetic, relational, and logical operations using Python operators. It shows how to perform basic mathematical calculations, compare values, and combine conditions using logical operators. This helps in understanding the use of different operators in Python programs.

Practical No. 4

Definition : Write a program to demonstrate string operations including slicing formatting and built-in string functions.

Aim/ Objective:

- To understand and perform basic string operations in Python.
- To learn how to access and manipulate strings using indexing and slicing.
- To understand string formatting and its use in Python programs.
- To use built-in string functions for performing various operations on strings.

- To develop a basic understanding of string manipulation in Python.

Source Code:

```
sample = "Python Programming is fun!"
```

```
print(f"\nOriginal String\t\t: ",sample)
print("First 6 character \t: ",sample[0:6])
print("Slice after index 7\t:",sample[7:])
```

```
name = "joshi"
marks = 98.552
```

```
F_string = f"Hello {name}, my marks is {marks:.2f}%."
print("F-string\t:",F_string)
```

```
Format = "Hello {}, my marks is {:.2f}%.".format(name,marks)
print(".format\t\t:",Format)
```

```
legacy = "Hello %s, my marks is %.2f%%."%(name,marks)
print("% Operator\t:",legacy)
```

```
text = "ArTiflcial IntElliGenCe"
```

```
print("\nText change to lower\t: ",text.lower())
print("Text change to upper\t: ",text.upper())
print("Check length of the text: ",len(text))
print("Replace new with text\t: ",text.replace("ArTiflcial","O"))
print("Capitalized\t\t: ",text.capitalize())
print("Title case\t\t: ",text.title())
```

```
print("Strip whitespace\t: ", " Python  ".strip())
print("Find the position of 'i': ", text.find("i"))
print("Count of the 'i'\t: ", text.count("i"))
print("Split the string\t:", text.split(" "))
```

Output:

Original String : Python Programming is fun!

First 6 character : Python

Slice after index 7 : Programming is fun!

F-string : Hello joshi, my marks is 98.55%.

.format : Hello joshi, my marks is 98.55%.

% Operator : Hello joshi, my marks is 98.55%.

Text change to lower : artificial intelligence

Text change to upper : ARTIFICIAL INTELLIGENCE

Check length of the text: 23

Replace new with text: O IntElliGenCe

Capitalized : Artificial intelligence

Title case : Artificial Intelligence

Strip whitespace : Python

Find the position of 'i': 3

Count of the 'i': 3

Split the string: ['ArTifIcial', 'IntElliGenCe']

Conclusion:

The program successfully demonstrates various string operations in Python, including indexing, slicing, formatting, and the use of built-in string functions. It shows how strings can be accessed, modified, formatted, and manipulated using different Python techniques.

Practical No. 5

Definition : Write a program to create and manipulate lists using indexing slicing and list comprehensions.

Aim/ Objective:

- To understand the concept of lists in Python.
- To learn how to create and manipulate lists.
- To access list elements using indexing.
- To perform operations on lists using slicing.
- To understand and use list comprehensions for creating lists efficiently.

Source Code:

```
list=[11,22,33,44,55,66,77]
```

```
print("Indexing: ")
```

```
print("First Element= ",list[0])
```

```
print("Last Element= ",list[-1])
```

```
print("Slicing: ")
```

```
print("First 3 Elements=",list[:3])
```

```
print("Last 3 Elements=",list[-3:])
```

```
print("Reverse List=",list[::-1])
```

```
print("List Manipulation: ")
```

```
list.append(88)
```

```
print("After Append= ",list)
```

```
list.remove(22)
```

```
print("After Remove= ",list)
```

```
print("List Comprehension: ")
```

```
square=[x*x for x in list]
```

```
print("Square List= ",square)
```

Output:

Indexing:

First Element= 11

Last Element= 77

Slicing:

First 3 Elements= [11, 22, 33]

Last 3 Elements= [55, 66, 77]

Reverse List= [77, 66, 55, 44, 33, 22, 11]

List Manipulation:

After Append= [11, 22, 33, 44, 55, 66, 77, 88]

After Remove= [11, 33, 44, 55, 66, 77, 88]

List Comprehension:

Square List= [121, 1089, 1936, 3025, 4356, 5929, 7744]

Conclusion:

The program successfully demonstrates how to create and manipulate lists in Python. It shows how to access elements using indexing, extract elements using slicing, and create new lists efficiently using list comprehensions. This helps in understanding the basic operations and flexibility of Python lists.

Practical No. 6

Definition : Write a program to illustrate the use of tuples and sets with basic operations.

Aim/ Objective:

- To understand the concept of tuples and sets in Python.
- To learn how to create and access elements of tuples.
- To perform basic operations on tuples and sets.
- To understand the properties and differences between tuples and sets.
- To learn how sets are used to store unique elements and perform set operations.

Source Code:

```
t1=('red','green',1,2,3,4)
```

```
t2=('apple','banana',90,'orange')
```



```
print("tuple 1:",t1[0])
print("tuple 2:",t1[1:3])
t3=t1+t2
print(t3)
print(len(t2))

set={'python',1,34,'language'}
print("set is ",set)
```

Output:

```
tuple 1: red
tuple 2: ('green', 1)
('red', 'green', 1, 2, 3, 4, 'apple', 'banana', 90, 'orange')
4
set is  {1, 'python', 'language', 34}
```

Conclusion:

The program successfully demonstrates the use of tuples and sets in Python. It shows how to create, access, and perform basic operations on tuples and sets. It also helps in understanding the differences between tuples and sets and their use for storing and managing data.

Practical No. 7

Definition : Write a program to create a dictionary and demonstrate dictionary methods and iteration.

Aim/ Objective:

- To understand the concept of dictionaries in Python.
- To learn how to create and access key-value pairs in a dictionary.
- To demonstrate various built-in dictionary methods.
- To learn how to add, update, and remove dictionary elements.
- To understand how to iterate through dictionary keys, values, and items.

Source Code:

```
dict={'name':'abc','age':8,'class':'MCA'}  
print(dict['name'],dict['age'],dict['class'])  
print(list(dict.values()))  
#print(dict['gender'])
```

Output:

```
abc 8 MCA  
['abc', 8, 'MCA']
```

Conclusion:

The program successfully demonstrates how to create and manipulate dictionaries in Python. It shows the use of different dictionary methods to add, update, and remove elements, and demonstrates how to iterate through keys, values, and key-value pairs. This helps in understanding the efficient use of dictionaries for storing and managing data.

Practical No. 8

Definition : Write a program to explain mutable and immutable objects in Python.

Aim/ Objective:

- To understand the concept of mutable and immutable objects in Python.
- To learn the difference between mutable and immutable data types.
- To demonstrate how mutable objects can be changed after creation.
- To demonstrate how immutable objects cannot be changed after creation.
- To understand the behavior of different Python objects during modification.

Source Code:

```
#immutable  
  
dict={'name':'abc','age':8,'class':'MCA'}  
print(dict['name'],dict['age'],dict['class'])  
print(list(dict.values()))  
#print(dict['gender'])  
#dict['gender']  
  
#mutable  
  
set1=set()  
  
set2={'python',556,879,'language'}
```

```
print(set2)
```

Output:

```
abc 8 MCA
```

```
['abc', 8, 'MCA']
```

```
{'language', 'python', 556, 879}
```

Conclusion:

The program successfully demonstrates the concept of mutable and immutable objects in Python. It shows that mutable objects can be modified after creation, while immutable objects cannot be changed. This helps in understanding how different Python data types behave when their values are modified.

Practical No. 9

Definition : Write a program to define and use user-defined functions with different types of arguments.

Aim/ Objective:

- To understand the concept of user-defined functions in Python.
- To learn how to define and call functions in a Python program.
- To understand different types of function arguments.
- To demonstrate the use of positional, keyword, default, and variable-length arguments.
- To understand how functions improve code reusability and organization.

Source Code:

```
def my_function():
```

```
    print('this is user defined function')
```

```
my_function()
```

```
def add(a,b):
```

```
    print(a+b)
```

```
    return
```

```
add(1,3)
```

```
def name(a='python'):
```

```
print(a)
return
#name(1)
name()
```

Output:

this is user defined function

4

python

Conclusion:

The program successfully demonstrates how to define and use user-defined functions in Python. It shows the use of different types of arguments, including positional, keyword, default, and variable-length arguments. This helps in understanding how functions make programs more organized, reusable, and easier to manage.

Practical No. 10

Definition : .Write a program to demonstrate recursion using factorial or Fibonacci series.

Aim/ Objective:

- To understand the concept of recursion in Python.
- To learn how a function can call itself to solve a problem.
- To demonstrate recursion using factorial or Fibonacci series.
- To understand the use of a base condition in recursive functions.
- To develop logical thinking and problem-solving skills using recursion.

Source Code:

```
def factorial(n):
    if n==0 or n==1:
        return 1
    else:
        return n * factorial(n-1)

#result=factorial(5)#5*4*3*2*1
```

```
#print('factorial is:',result)
```

```
def fib(n):  
    if n<=1:  
        return n  
    return fib(n-1) + fib(n-2)  
num=9  
for i in range(num):  
    print(fib(i),end=" ")
```

Output:

0 1 1 2 3 5 8 13 21

Conclusion:

The program successfully demonstrates the concept of recursion in Python using factorial or Fibonacci series. It shows how a function can call itself repeatedly until a base condition is reached. This helps in understanding recursive functions and their use in solving problems efficiently.